Boardroom: Every Folder and File Explained

This document lists every folder and every file in the repository and says what it is for. Paths are relative to the repository root. The application itself lives in `whiteboard-code-verifier/`; everything above it is documentation and deployment wrappers.

---

1. Repository root

File or folder	What it is
`README.md`	Main project page on GitHub: what Boardroom is, architecture, security, tools, how to run it, limits
`.gitignore`	Tells Git which files never to commit (dependencies, Python environments, logs, secrets)
`Dockerfile`	Builds the whole app into one container when deploying from the repository root (used by Railway)
`package.json`	Root wrapper so hosting services can build and start the app from the repository root
`railway.json`	Railway hosting settings: build with the root `Dockerfile`, start the Node server, restart on failure
`docs/`	Project-level documents (see section 2)
`.vibe-wise/`	Learning notes kept by a tutoring assistant (see section 3)
`whiteboard-code-verifier/`	The application (sections 4 onwards)

2. `docs/`: project documents

File	What it is
`PRD.md`	Technical product requirements document: goals, user stories, architecture, API, security, configuration
`PRD.pdf`	PDF version of the PRD
`PROJECT_REPORT.md`	Plain-language explanation of the project, the tools used and the security measures
`FUNCTIONS.pdf`	Every function in the source code with a one-line explanation
`FILES.md`	This document
`ORIGINAL_NOTES.md`	The original concept notes and design history from before the app was built

3. `.vibe-wise/`: learning notes

File	What it is
`profile.md`	The learner's experience level and preferences, used to pitch explanations
`progress.md`	Record of concepts explained and design decisions made
`project-map.md`	Short summary of the project's purpose, components and main flow

These notes are not part of the application and are never loaded by it.

---

4. `whiteboard-code-verifier/`: application root

File	What it is
`package.json`	The app's npm settings: dependencies (React, Excalidraw, Vite, Playwright) and the commands (`dev`, `build`, `start`, `test`, `runner:build` and others)
`package-lock.json`	Exact versions of every dependency, so installs are identical everywhere
`index.html`	The single web page; it loads `src/main.jsx`
`vite.config.js`	Vite settings: the development server and the proxy that forwards `/api` requests to the Node API
`vercel.json`	Vercel hosting settings: time limits and extra files for each serverless function
`playwright.config.js`	Browser-test settings: starts the server on port 8878 and runs the tests in Chromium
`Dockerfile`	Builds the app into a container when the app folder itself is deployed
`.env.example`	Template listing every setting the server reads (owner key, OCR address, archive, Wandbox); copy it to `.env`
`.gitignore`	Keeps `.env`, `node_modules/`, `dist/`, `snapshots/` and test output out of Git
`README.md`	Detailed app guide: features, local setup, handwriting, languages, lesson history, verification

5. `server/`: the Node API

The backend. Every browser request arrives at `index.mjs`, passes the security checks, and is handed to the right module.

File	What it is
`index.mjs`	Entry point. Starts the HTTP server on port 8787, applies security headers, host/origin checks, the owner key and rate limits, routes each `/api` URL, and serves the built website

`server/security/`: who may call the API and how often

File	What it is
`owner-key.mjs`	Checks the owner key so only the key holder can use the API when `BOARDROOM_OWNER_KEY` is set
`rate-limit.mjs`	Per-visitor and server-wide request limits per minute

`server/execution/`: where code is run

File	What it is
`sandbox.mjs`	Starts a fresh, locked-down Docker container for each run and always removes it afterwards
`hosted-runner.mjs`	Sends code to the Wandbox web service instead, when Docker is not available (hosted mode)

`server/ai/`: the model client

File	What it is
`ocr.mjs`	Validates handwriting images and sends them to the local Qwen2-VL OCR service; also loads `.env`
`snippet.mjs`	Snippet mode: asks the model to add boilerplate and checks that the teacher's lines were kept

`server/lessons/`: lesson history

File	What it is
`snapshots.mjs`	Saves each lesson to a local Git repository and lists, reads and compares saved lessons
`github-snapshots.mjs`	The same features stored in a GitHub repository instead (hosted mode)
`snapshot-bundle.mjs`	Builds the replay bundle saved with each lesson: frozen runner, headers, Dockerfile and SHA-256 manifest
`snapshot-sync.mjs`	Pushes local lesson commits to a remote repository in the background
`snapshot-replay.py`	Script stored inside every lesson that verifies the files and replays the run in Docker

`server/hosted/`: Vercel support

File	What it is
`hosted-api.mjs`	Shared helpers for the Vercel functions: origin and owner-key checks, JSON body parsing, error responses

6. `api/`: Vercel serverless functions

Thin wrappers that let Vercel run the server code as separate functions. Each file is one URL.

File	What it is
`health.js`	`GET /api/health`: what is available and whether the caller holds the owner key
`run.js`	`POST /api/run`: runs code on Wandbox
`ocr.js`	`POST /api/ocr`: handwriting recognition through a hosted OCR service
`snippet.js`	`POST /api/snippet`: Snippet mode
`snapshots.js`	Lesson history stored in GitHub
`session.js`	`/api/session`: kept so older browsers clear their old session cookie

7. `shared/`: code used by both browser and server

File	What it is
`board.js`	Reads frames, code (black) and input (blue) out of whiteboard elements; builds program fingerprints and line maps
`handwriting.js`	Detects handwriting in a frame, fingerprints the ink and builds the OCR review draft
`languages.js`	The supported languages (C++ and Python) and the request fields for each
`libraries.js`	The catalogue of teaching headers and standard-library examples shown in the Libraries tab

8. `src/`: the website

File	What it is
`main.jsx`	The application shell: all state, the run button, toolbar, board tabs and side panel
`styles.css`	All styling, including dark mode

`src/components/`: building blocks of the interface

File	What it is
`CodeEditor.jsx`	The editable "code that runs" box with error underlines
`DrawToolbar.jsx`	The drawing toolbar: pen, eraser, hand, select, shapes, widths and colours
`FrameControls.jsx`	Width and height controls for the selected program frame
`HandwritingPanel.jsx`	The Handwriting tab: reads ink with the model and lets the teacher correct and confirm it
`Icon.jsx`	The small line icons used across the app
`LibraryPanel.jsx`	The Libraries tab: header list, source and an example loader
`MemoryDiagram.jsx`	Diagram of the stack and heap addresses printed by the memory lesson
`OutputPanel.jsx`	The Output tab: progress, result, stdout, diagnostics and teaching notes
`OwnerKeyForm.jsx`	The owner-key box shown when the API is private
`ProgramRegions.jsx`	Number labels drawn over program regions on the board
`SnapshotHistory.jsx`	The History tab: saved lessons, opening them and sync status
`SnippetReview.jsx`	Snippet mode: shows the prepared program and the Confirm & run button
`TestCases.jsx`	The Cases tab: up to 20 inputs with expected results
`WorkspaceLayout.jsx`	Splits the screen into the board and a resizable side panel

`src/lib/`: browser helpers without any interface

File	What it is
`api.js`	The one place the browser calls the API; attaches the owner key and never sends cookies
`api-response.js`	Parses API replies safely and turns hosting errors into readable messages
`canvas.js`	Changes the board: loads lessons, draws error highlights, resizes and fits frames
`capture.js`	Exports handwritten ink as PNG images for OCR
`lessons.js`	The six built-in demo lessons
`linked-list-drawing.js`	Turns a drawn chain such as `1->3->4` into a real C++ linked-list program
`test-cases.js`	Rules for test cases: validation, storage and output comparison

9. `runner/`: what runs inside the Docker sandbox

File	What it is
`Dockerfile`	Builds the runner image `whiteboard-runner:local` (gcc 14.2 and Python 3, unprivileged user)
`runner.py`	Runs inside the container: checks includes, compiles with sanitizers, runs with limits, returns JSON results and teaching notes
`lib/stack.h`	Teaching header: an integer stack
`lib/queue.h`	Teaching header: a growing circular queue
`lib/linked_list.h`	Teaching header: a singly linked list, including `list_from_arrows`
`lib/search.h`	Teaching header: binary search with lower and upper bounds
`lib/board_contract.h`	Shared helper the teaching headers use to report misuse at the teacher's line

10. `ocr-service/`: the AI model server

File	What it is
`service.py`	Python server on port 8788: loads Qwen2-VL once and answers `/ocr` (read handwriting), `/wrap` (Snippet mode) and `/health`
`requirements.txt`	Python libraries it needs (PyTorch, Transformers, Pillow, NumPy)
`Dockerfile`	Builds the service into a container with the model weights included
`README.md`	How to install, run and configure the service, and its limits

11. `scripts/`: developer commands

File	What it is
`dev.mjs`	What `npm run dev` runs: starts the API and the Vite website together
`fonts.mjs`	Copies Excalidraw's fonts into `public/fonts` before a build
`check-runner.mjs`	Smoke test of the real Docker runner with the demo lessons
`check-ocr.mjs`	Sends one PNG through the running OCR service and prints the result
`benchmark-ocr.mjs`	Measures OCR error rates on a set of private sample images
`ocr-benchmark-metrics.mjs`	The error-rate calculations used by the benchmark

12. `public/`

File	What it is
`favicon.svg`	The browser tab icon

13. `docs/` (inside the app): technical guides

File	What it is
`technical-overview.md`	Full technical description: stack, data flow, OCR, execution limits, API, storage
`ocr-options.md`	Which handwriting models were considered and why Qwen2-VL was chosen
`c-libraries.md`	Reference for the teaching headers: functions, rules and examples
`docker-api-deployment.md`	Hosting the API on your own server behind Caddy
`vercel-serverless.md`	Hosting on Vercel and the settings it needs

14. `tests/`: automatic checks

Unit tests (run with `npm test`)

File	What it checks
`api.test.mjs`	The API handler: routing, limits, rate limits, owner key, error responses
`api-response.test.mjs`	Browser-side parsing of API replies and hosting errors
`board.test.mjs`	Reading code and input from board elements
`handwriting.test.mjs`	Handwriting detection, fingerprints and review drafts

File	What it checks
`ocr.test.mjs`	The OCR client: image validation, timeouts, error handling
`snippet.test.mjs`	Snippet mode: the line-preservation check and model errors
`rate-limit.test.mjs`	Per-visitor and server-wide limits
`hosted-api.test.mjs`	The Vercel functions' access checks
`hosted-runner.test.mjs`	Running code through Wandbox
`snapshots.test.mjs`	Saving lessons to local Git and verifying replay bundles
`github-snapshots.test.mjs`	Saving lessons to GitHub
`linked-list-drawing.test.mjs`	Turning drawn chains into programs
`test-cases.test.mjs`	Test-case validation and output comparison
`ocr-benchmark.test.mjs`	The benchmark's error-rate maths
`test_runner.py`	The container runner: compilation, limits, diagnostics
`test_libraries.py`	The teaching headers' behaviour
`test_flow.py`	End-to-end runner flows

Browser tests in `tests/browser/` (run with `npm run test:browser`)

File	What it checks
`workspace.spec.js`	Canvas editing, download/import and real Docker runs of every demo lesson
`handwriting.spec.js`	Pen capture, review, confirmation and running confirmed code
`selection.spec.js`	Selecting code regions, whole page, zoom and panel resizing
`cases.spec.js`	Running several test cases
`python.spec.js`	Python execution and Python handwriting
`snapshots.spec.js`	Lesson history in the browser
`automatic-snapshots.spec.js`	Automatic saving after each run
`layout.spec.js`	Layout on different screen sizes
`session.spec.js`	Old session cookies being cleared
`api-access.spec.js`	Behaviour when the API is remote or unavailable
`helpers.js`, `board-helpers.js`	Shared helpers the browser tests use to build boards and drive the page

Fixtures in `tests/fixtures/`

File	What it is
`library_checks.c`	C program that exercises the teaching headers
`ocr/synthetic-code.png`, `ocr/synthetic-python.png`	Sample handwriting images
`ocr/synthetic-code.expected.c`, `ocr/synthetic-python.expected.txt`	The exact text those images contain
`ocr/manifest.json`	Lists the samples for the benchmark
`ocr/README.md`	How to use the samples

---

15. Folders on your disk that are not in Git

These are created by commands and kept out of the repository by `.gitignore`.

Folder	Created by	Delete it?
`node_modules/`	`npm ci`	Only to reinstall; the app needs it
`ocr-service/.venv/`	`python -m venv` and `pip install`	Only to reinstall; the model needs it
`dist/`	`npm run build`	Safe; rebuilt by the next build
`public/fonts/`	`npm run build`	Safe; rebuilt automatically
`test-results/`	Browser tests	Safe
`snapshots/`	Saving lessons	**No: these are your saved lessons**
`.env`	You, from `.env.example`	**No: holds your private settings**